```
    inv.open("inv", ios::in | ios::binary);

    if(!inv)
    {
        cout << "Cannot open file for input.\n";
        return 1;
    }

    do
    {
        cout << "Record # (-1 to quit): ";
        cin >> i;
        if(i == -1)
            break;
        inv.seekg(i*(SIZE+sizeof(int)+sizeof(double)), ios::
           beg);
        temp.retrieve(inv);
        cout << temp;
    }
    while(inv.good());

    inv.close();

    return 0;
}
```

REVIEW SKILLS CHECK: Chapter 10

1. ```
   #include <iostream>
   using namespace std;

   ostream &setsci(ostream &stream)
   {
       stream.setf(ios::scientific | ios::uppercase);

       return stream;
   }

   int main()
   {
       double f = 123.23;

       cout << setsci << f;
       cout << '\n';

       return 0;
   }
   ```

2. ```
   // Copy and convert tabs to spaces.
   #include <iostream>
   ```

```
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    if(argc!=3)
    {
        cout << "Usage: CPY <int> <out>\n";
        return 1;
    }

    ifstream in(argv[1]);
    if(!in)
    {
        cout << "Cannot open input file.\n";
        return 1;
    }

    ofstream out(argv[2]);
    if(!out)
    {
        cout << "Cannot open output file.\n";
        return 1;
    }

    char ch;
    int i = 8;

    while(!in.eof())
    {
        in.get(ch);
        if(ch=='\t')
            for( ; i>0; i--)
                out.put(' ');
        else
            out.put(ch);
        if(i == -1 || ch=='\n')
            i = 8;
        i--;
    }

    in.close();
    out.close();

    return 0;
}
```

3.
```
// Search file.
#include <iostream>
#include <fstream>
#include <cstring>
```

```
using namespace std;

int main(int argc, char *argv[])
{
    if(argc!=3)
    {
        cout << "Usage: SEARCH <file> <word>\n";
        return 1;
    }

    ifstream in(argv[1]);
    if(!in)
    {
        cout << "Cannot open input file.\n";
        return 1;
    }

    char str[255];
    int count=0;

    while(!in.eof())
    {
        in >> str;
        if(!strcmp(str, argv[2]))
            count++;
    }

    cout << argv[2] << " found " << count;
    cout << " number of times.\n";

    in.close();

    return 0;
}
```

4. The statement is

```
out.seekp(234, ios::beg);
```

5. The functions are **rdstate()**, **good()**, **eof()**, **fail()**, and **bad()**.

6. The C++ I/O system can be customized to operate on classes that you create.

10.2 EXERCISES

1.
```
#include <iostream>
using namespace std;

class num
{
public:
    int i;
```

```
    num(int x) { i = x; }
    virtual void shownum() { cout << i << '\n'; }
};

class outhex : public num
{
public:
    outhex(int n) : num(n) {}
    void shownum() { cout << hex << i << '\n'; }
};

class outoct : public num
{
public:
    outoct(int n) : num(n) {}
    void shownum() { cout << oct << i << '\n'; }
};

int main()
{
    outoct o(10);
    outhex h(20);

    o.shownum();
    h.shownum();

    return 0;
}
```

2.
```
#include <iostream>
using namespace std;

class dist
{
public:
    double d;
    dist(double f) { d = f; }
    virtual void trav_time()
    {
        cout << "Travel time at 60 mph: ";
        cout << d / 60 << '\n';
    }
};

class metric : public dist
{
public:
    metric(double f) : dist(f) {}
    void trav_time()
    {
        cout << "Travel time at 100 kph: ";
```

```
        cout << d / 100 << '\n';
    }
};

int main()
{
    dist *p, mph(88.0);
    metric kph(88);

    p = &mph;
    p->trav_time();

    p = &kph;
    p->trav_time();

    return 0;
}
```

10.3 EXERCISES

2. By definition, an abstract class contains at least one pure virtual function. This means that no body for that function exists relative to that class. Thus, there is no way that an object can be created because the class definition is not complete.

3. When **func()** is called relative to **derived1**, it is the **func()** inside **base** that is used. The reason this works is that virtual functions are hierarchical.

10.4 EXERCISES

1. ```
   // Demonstrate virtual functions.
   #include <iostream>
   #include <cstdlib>
   using namespace std;

   class list
   {
   public:
       list *head; // pointer to start of list
       list *tail; // pointer to end of list
       list *next; // pointer to next item
       int num; // value to be stored

       list() { head = tail = next = NULL; }
       virtual void store(int i) = 0;
       virtual int retrieve() = 0;
   };

   // Create a queue-type list.
   class queue : public list
   {
   public:
   ```
   ```

```
    void store(int i);
    int retrieve();
};

void queue::store(int i)
{
    list *item;

    item = new queue;
    if(!item)
    {
        cout << "Allocation error.\n";
        exit(1);
    }
    item->num = i;

    // put on end of list
    if(tail)
        tail->next = item;
    tail = item;
    item->next =  NULL;
    if(!head)
        head = tail;
}

int queue::retrieve()
{
    int i;
    list *p;

    if(!head)
    {
        cout << "List empty.\n";
        return 0;
    }

    // remove from start of list
    i = head->num;
    p = head;
    head = head->next;
    delete p;

    return i;
}

// Create a stack-type list.
class stack : public list
{
public:
    void store(int i);
    int retrieve();
```

```
};

void stack::store(int i)
{
    list *item;

    item = new stack;
    if(!item)
    {
        cout << "Allocation error.\n";
        exit(1);
    }
    item->num = i;

    // put on front of list for stack-like operation
    if(head)
        item->next = head;
    head = item;
    if(!tail)
        tail = head;
}

int stack::retrieve()
{
    int i;
    list *p;

    if(!head)
    {
        cout << "List empty.\n";
        return 0;
    }

    // remove from start of list
    i = head->num;
    p = head;
    head = head->next;
    delete p;

    return i;
}

// Create a sorted list.
class sorted : public list
{
public:
    void store(int i);
    int retrieve();
};

void sorted::store(int i)
```

```
{
    list *item;
    list *p, *p2;

    item = new sorted;
    if(!item)
    {
        cout << "Allocation error.\n";
        exit(1);
    }
    item->num = i;

    // find where to put next item
    p = head;
    p2 = NULL;
    while(p) // goes in middle
    {
        if(p->num > i)
        {
            item->next = p;
            if(p2)
                p2->next = item; // not 1st element
            if(p==head)
                head = item; // new first element
            break;
        }
        p2 = p;
        p = p->next;
    }
    if(!p) // goes on end
    {
        if(tail)
            tail->next = item;
        tail = item;
        item->next = NULL;
    }
    if(!head) // is first element
        head = item;
}

int sorted::retrieve()
{
    int i;
    list *p;

    if(!head)
    {
        cout << "List empty.\n";
        return 0;
    }
```

```
    // remove from start of list
    i = head->num;
    p = head;
    head = head->next;
    delete p;

    return i;
}

int main()
{
    list *p;

    // demonstrate queue
    queue q_ob;
    p = &q_ob; // point to queue

    p->store(1);
    p->store(2);
    p->store(3);

    cout << "Queue: ";
    cout << p->retrieve();
    cout << p->retrieve();
    cout << p->retrieve();

    cout << '\n';

    // demonstrate stack
    stack s_ob;
    p = &s_ob; // point to stack

    p->store(1);
    p->store(2);
    p->store(3);

    cout << "Stack: ";
    cout << p->retrieve();
    cout << p->retrieve();
    cout << p->retrieve();

    cout << '\n';

    // demonstrate sorted list
    sorted sorted_ob;
    p = &sorted_ob;

    p->store(4);
    p->store(1);
    p->store(3);
    p->store(9);
```

```
    p->store(5);

    cout << "Sorted: ";
    cout << p->retrieve();
    cout << p->retrieve();
    cout << p->retrieve();
    cout << p->retrieve();
    cout << p->retrieve();

    cout << '\n';

    return 0;
}
```

MASTERY SKILLS CHECK: Chapter 10

1. A virtual function is essentially a placeholder function that is declared in a base class and that is redefined by a class derived from that base. The process of redefinition is called overriding.

2. Nonmember functions and constructor functions cannot be made virtual.

3. A virtual function supports run-time polymorphism through the use of base class pointers. When a base class pointer points to an object of a derived class containing a virtual function, the specific function called is determined by the type of object being pointed to.

4. A pure virtual function is one that contains no definition relative to the base class.

5. An abstract class is a base class that contains at least one pure virtual function. A polymorphic class is one that contains at least one virtual function.

6. The fragment is incorrect because the redefinition of a virtual function must have the same return type and type and number of parameters as the original function. In this case, the redefinition of **f()** differs in the number of its parameters.

7. Yes.

CUMULATIVE SKILLS CHECK: Chapter 10

1.
```
// Demonstrate virtual functions.
#include <iostream>
#include <cstdlib>
using namespace std;

class list
{
public:
    list *head; // pointer to start of list
    list *tail; // pointer to end of list
    list *next; // pointer to next item
    int num; // value to be stored
```

```
    list() { head = tail = next = NULL; }
    virtual void store(int i) = 0;
    virtual int retrieve() = 0;
};

// Create a queue-type list.
class queue : public list
{
public:
    void store(int i);
    int retrieve();
    queue operator+(int i) { store(i); return *this; }
    int operator--(int unused) { return retrieve(); }
};

void queue::store(int i)
{
    list *item;

    item = new queue;
    if(!item)
    {
        cout << "Allocation error.\n";
        exit(1);
    }
    item->num = i;

    // put on end of list
    if(tail)
        tail->next = item;
    tail = item;
    item->next =  NULL;
    if(!head)
        head = tail;
}

int queue::retrieve()
{
    int i;
    list *p;

    if(!head)
    {
        cout << "List empty.\n";
        return 0;
    }

    // remove from start of list
    i = head->num;
    p = head;
    head = head->next;
```

```
    delete p;

    return i;
}

// Create a stack-type list.
class stack : public list
{
public:
    void store(int i);
    int retrieve();
    stack operator+(int i) { store(i); return *this; }
    int operator--(int unused) { return retrieve(); }
};

void stack::store(int i)
{
    list *item;

    item = new stack;
    if(!item)
    {
        cout << "Allocation error.\n";
        exit(1);
    }
    item->num = i;

    // put on front of list for stack-like operation
    if(head)
        item->next = head;
    head = item;
    if(!tail)
        tail = head;
}

int stack::retrieve()
{
    int i;
    list *p;

    if(!head)
    {
        cout << "List empty.\n";
        return 0;
    }

    // remove from start of list
    i = head->num;
    p = head;
    head = head->next;
    delete p;
```

```
    return i;
}

int main()
{
    // demonstrate queue
    queue q_ob;

    q_ob + 1;
    q_ob + 2;
    q_ob + 3;

    cout << "Queue: ";
    cout << q_ob--;
    cout << q_ob--;
    cout << q_ob--;

    cout << '\n';

    // demonstrate stack
    stack s_ob;

    s_ob + 1;
    s_ob + 2;
    s_ob + 3;

    cout << "Stack: ";
    cout << s_ob--;
    cout << s_ob--;
    cout << s_ob--;

    cout << '\n';

    return 0;
}
```

2. Virtual functions differ from overloaded functions in that overloaded functions *must* differ from the number of parameters or the type of parameters. An overridden virtual function must have exactly the same prototype (that is, the same return type and the same type and number of parameters) as the original function.

REVIEW SKILLS CHECK: Chapter 11

1. A virtual function is a function that is declared as **virtual** by the base class and then overridden by a derived class. A base class that contains at least one pure virtual function is called an abstract class.

2. The missing words are "virtual" and "base".

3. If a derived class does not override a non-pure virtual function, the derived class will use the base class's version of the virtual function.

4. The main advantage of run-time polymorphism is flexibility. The main disadvantage is loss of execution speed.

11.1 EXERCISES

2.
```
#include <iostream>
using namespace std;

template <class X> X min(X a, X b)
{
    if(a<=b)
        return a;
    else
        return b;
}

int main()
{
    cout << min(12.2, 2.0);
    cout << endl;
    cout << min(3, 4);
    cout << endl;
    cout << min('c', 'a');

    return 0;
}

/*
   This min(X, X) function might cause ambiguity
   because of STL function min(const _Tp&, const _Tp&).
   Try a different name.
*/
```

3.
```
#include <iostream>
#include <cstring>
using namespace std;

template <class X> int find(X object, X *list, int size)
{
    int i;

    for(i=0; i<size; i++)
        if(object == list[i])
            return i;
    return -1;
}

int main()
{
    int a[] = {1, 2, 3, 4};
    char *c = "this is a test";
```